FTRIA-004 — Constraint Surfaces and Invariant Boundaries

Part I — Foundations of Runtime-Invariant Degrees of Freedom

Sizhe Tan
with chatGPT (OpenAI)
2026-07-15

Repository: https://github.com/sizhet/Function-Tunnel-and-Runtime-Invariant-Algebra-FTRIA

Abstract

Runtime Invariants possess Degrees of Freedom, allowing implementations to evolve while preserving invariant identity.
However, these freedoms are not unlimited.
Every Runtime Invariant is surrounded by structural constraints that define the boundary between identity preservation and identity transition.
This paper introduces two fundamental concepts:

- **Constraint Surfaces**, which describe the structural limits governing admissible runtime evolution.
- **Invariant Boundaries**, which separate implementations representing the same Runtime Invariant from those representing different Runtime Invariants.

Together, these concepts establish the geometric foundation of Runtime-Invariant Degrees of Freedom and provide the basis for Runtime Geometry and future Runtime Algebra.

1. Degrees of Freedom Require Boundaries

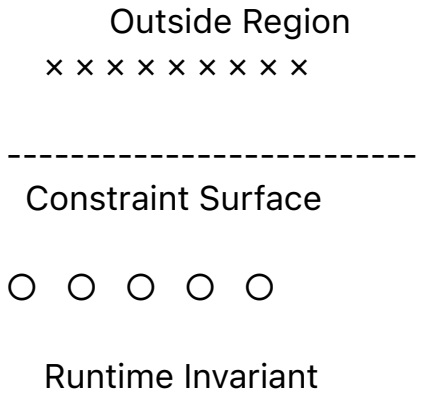
The previous papers established that Runtime Invariants permit admissible variation.
Without constraints, however, the concept of admissibility becomes meaningless.
Degrees of Freedom therefore imply the existence of boundaries.
Every Runtime Invariant answers two complementary questions:

- What changes are allowed?
- What changes terminate invariant identity?

The first defines freedom.
The second defines the boundary.

2. Constraint Surfaces

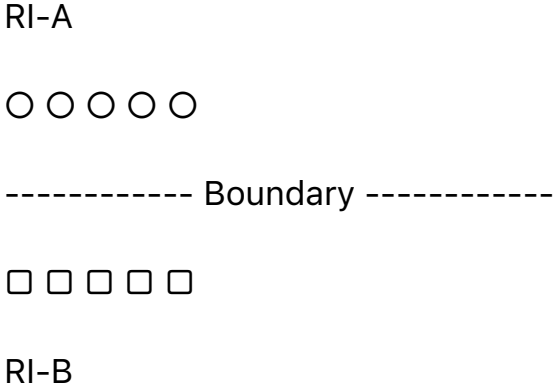
A **Constraint Surface** is the structural limit that separates admissible runtime variations from non-admissible ones.
Conceptually,



Implementations located beneath the surface preserve the Runtime Invariant.
Crossing the surface changes the Runtime Invariant itself.
Constraint Surfaces therefore define the operational limits of invariant-preserving evolution.

3. Invariant Boundaries

The **Invariant Boundary** represents the transition between two Runtime Invariants.



Movement inside one side preserves identity.
Crossing the boundary produces another Runtime Invariant.
The boundary is therefore not merely an implementation difference.
It represents a change of structural identity.

4. Boundaries Are Structural Rather Than Physical

Invariant Boundaries should not be confused with

- programming-language boundaries,

- module boundaries,
- file boundaries,
- process boundaries,
- hardware boundaries.

Instead,
they emerge from structural capability.
Two implementations may differ dramatically in code organization while remaining within the same Runtime Invariant.
Conversely,
a seemingly minor modification may alter the structural capability sufficiently to cross an Invariant Boundary.
Therefore,
boundaries are determined by preserved structural identity rather than implementation appearance.

5. Constraint Surfaces May Be High-Dimensional

Real Runtime Invariants rarely possess a single limiting condition.
Instead,
multiple structural constraints interact simultaneously.
Examples include

- functional correctness,
- calling graph consistency,
- dependency preservation,
- resource constraints,
- scheduling constraints,
- semantic consistency,
- runtime safety,
- architectural compatibility.

Collectively,
these constraints form a high-dimensional Constraint Surface surrounding the Runtime Invariant.
This surface defines the complete admissible region.

6. Runtime Evolution as Navigation

Software evolution can therefore be interpreted geometrically.
Instead of asking
"Can this implementation be modified?"
FTRIA asks
"Can the implementation move while remaining inside the Constraint Surface?"

Runtime optimization,
migration,
refactoring,
translation,
parallelization,
and deployment
all become navigation problems within admissible regions.
The engineering objective shifts from preserving code to preserving structural position.

7. Relationship to Previous Principles

The previous papers introduced

- Runtime Invariant Degrees of Freedom,
- RI-Preserving Transformations,
- Admissible Runtime Variation.

Constraint Surfaces provide the structural mechanism that governs these concepts.
Degrees of Freedom define where movement is possible.
RI-Preserving Transformations describe how movement occurs.
Admissible Runtime Variation identifies valid positions.
Constraint Surfaces determine the permissible region.
Invariant Boundaries define the transition between Runtime Invariants.
Together,
these concepts establish the geometric language of FTRIA.

8. Toward Runtime Geometry

Once Constraint Surfaces are established,
many advanced concepts naturally emerge.
These include

- structural stability,
- reachable runtime regions,
- deformation of Runtime Invariants,
- topology of invariant spaces,
- optimization trajectories,
- Runtime Configuration Spaces,
- Runtime Algebra.

FTRIA therefore extends software engineering from implementation management toward geometric reasoning over Runtime Invariants.

Key Takeaways

- Degrees of Freedom necessarily imply structural constraints.
- Constraint Surfaces define the admissible region of a Runtime Invariant.
- Invariant Boundaries separate different Runtime Invariants.
- Boundaries are determined by structural capability rather than implementation similarity.
- Runtime evolution can be interpreted as navigation within admissible regions.
- Constraint Surfaces establish the geometric foundation for Runtime Geometry and Runtime Algebra.

Related FTRIA Documents

- **FTRIA-001 — Runtime Invariant Degrees of Freedom**
- **FTRIA-002 — RI-Preserving Transformation**
- **FTRIA-003 — Admissible Runtime Variation**
- **FTRIA-005 — Local and Global Degrees of Freedom**